GAGNAÞON HAGSTOFU ÍSLANDS · 30 OCT – 1 NOV 2026

Spurðu Hagstofuna: the whole project explained

What we are building, how every part works and why it was built that way, what we measured, which ideas failed, where we stand two days before the event, and what is left to do.

Contents

1. The short version
2. The challenge
3. Our approach and its principles
4. One question, end to end
5. The parts in depth
6. The language models
7. How we measure
8. Experiment log
9. Hardware and speed
10. Why Track 2
11. Where we stand and what is missing
12. Plan, split and timeline
13. Lessons learned
14. Glossary
15. Where things are in the code
16. Appendix: the full diagram

Repository: github.com/Atlimh1/hagstofan (newest code on branch claude/wonderful-heisenberg-s3mbqr). State as of 29 October 2026. Every number in this document comes from the repository's evaluation scripts; the history is in CLAUDE.md, tasks/todo.md and eval/experiments/.

1 The short version

We are building a chatbot that answers questions about Statistics Iceland's data in plain Icelandic or English. It searches all ~2,100 tables in Hagstofa's PX-Web database, picks the right one, chooses the exact values (municipality, year, sex…), fetches the numbers **live** from the PX-Web API, does any arithmetic in code, and writes a short answer with a source that code adds, so it cannot be made up.

- **Search is our strength:** the right table is among the top 8 candidates for **86%** of 270 held-out questions. A search that uses only embeddings, like the organizers' starter kit, gets 65%.
- **The language model is solved for the core task:** given the right table, gemma4:26b chose the right values and wrote the right number in **24 of 24** test questions, with no invented numbers.
- **The weak step is picking the table**, for every model we tried, and for the organizers' starter kit too.
- **Just built, not yet measured with a real model:** calculations ("how much did it grow?") and questions that need two tables. These are 24 of the organizers' 52 test questions.
- **Still missing:** a FastAPI server (required), units in answers, showing the values used, stated assumptions, response time, and English answers to English questions.
- **Our route is Track 2** (direct PX-API), with fine-tuning of the search model on LUMI as a Track 3-style extension.

2 The challenge

Statistics Iceland's third AI data hackathon (Gagnaþon) asks for "a working prototype of an AI-powered statistical search engine": it takes a user's question, finds the right table and variables, retrieves the figures, and gives a clear answer with a transparent reference to the official statistics.

Rules that shape our design

- **Only freeware may run the solution:** no paid hosted LLM APIs, so open-weight models on the provided hardware (DGX Spark or the LUMI supercomputer). The solution must also be free to use.
- **AI coding assistants are allowed**, but the organizers review each person's and the team's own design and implementation work. Both of us must be able to explain every design choice.
- **Both tracks 1 and 2 must deliver a web chatbot connected to FastAPI**, testable with cURL, with clear loading and error messages, and answers that show values, units and sources.
- **"Let code do the maths, never the model"**: the organizers' own principle (their starter model wrote 1,024 for 1,032).

The three tracks

TRACK	STARTING POINT FROM THE ORGANIZERS	FOCUS
1 Prepared RAG	FastAPI + ChromaDB (migration data only) + the CPI from an Excel file + Llama 3 through Ollama	Interface and integration
2 Direct PX-API	One FastAPI file: three hardcoded tables (CPI, population, migration) chosen by keywords; fixed answer templates; no LLM	**Data selection and backend logic**
3 RAG on HPC	An embedding project on LUMI	Retrieval quality and performance

How it is judged (draft)

Six criteria, 1–5 points each (6–30 total): **functionality** (correct retrieval and answers), **presentation** (clear interface), **usability** (easy questions and navigation), **clarity** (understandable answers and sources), **deployment** (reproducible solution) and **user value** (a useful use case). "Assess the result and evidence, not the amount of hardware used."

The demo is the same for everyone, 10 minutes: (1) a real question in the web UI with the number, its unit and source; (2) explain the route; (3) evidence: check an answer against the table, an unclear question, the response time and a known limitation. Hand-in: the prototype, 5–7 slides, and a short write-up in the repository (what changed, before and after with their `gagnvist evaluate`, what blocked us, what's next). Five teams are shortlisted; the winner presents at the Gagnvist conference on 26 November.

The organizers' test questions

Their starter kit ships 52 test questions with known answers: **28 easy** (one number from one table), **12 medium** (a calculation in one table, e.g. change or share), **12 hard** (numbers from two tables); half in English. The starter kit gets 7 of 52 right; it reaches a right table for only 8 of the 28 easy ones.

3 Our approach and its principles

A pipeline of small steps, each done by whatever is best at it: the language model where judgement or language is needed, code where something must be exact, and Miðeind's Icelandic tools where Icelandic grammar matters.

PRINCIPLE	WHAT IT MEANS IN PRACTICE
The model chooses; code checks, computes and cites	The LLM picks the table, the value codes, the operation and the wording. Code validates every code against the table, does all arithmetic and adds the source.
Live numbers	Numbers are fetched from PX-Web at question time, never from the model's memory or a stale copy, so answers follow new publications.
Icelandic first	Base forms of words (BÍN), spelling correction (GreynirCorrect) and place categories make "í Kópavogi" find "Kópavogsbær".
Measure every change	Decide on a dev set, confirm once on a frozen test set, treat differences under ~3 points as noise, and read individual failures, not just totals.
The model is a setting	Any OpenAI-compatible server. Moving from a home PC to a DGX Spark or LUMI changes one variable.

4 One question, end to end

The organizers' own example from the info meeting, as our pipeline handles it. The numbers are theirs; the answer wording is illustrative, since this exact question has not been run through the pipeline with a real model yet.

QUESTION

Hversu mikið fjölgaði íbúum Reykjavíkur frá 2025 til 2026?

SPELLING

Already correct; GreynirCorrect leaves it unchanged.

SPLIT?

One topic, one table: kept as one part.

SEARCH, TOP 8

`MAN02005` "Mannfjöldi eftir kyni, aldri og sveitarfélögum 1998–2026" ranks high: "Reykjavík" boosts tables containing it, and the checker confirms it has 2025 and 2026.

LLM PICKS THE TABLE

MAN02005

LLM CHOOSES VALUES

`{"Sveitarfélag": ["0000"], "Ár": ["2025", "2026"]}`: both years, because the question compares them.

CODE VALIDATES

Both codes exist. Sex and age weren't chosen, so code fills in the totals ("Alls").

PX-WEB, LIVE

Reykjavíkurborg 2025: 138.772 · Reykjavíkurborg 2026: 139.804

LLM NAMES THE OPERATION

`{"operation": "difference", "a": 1, "b": 0}`

CODE COMPUTES

139.804 − 138.772 = 1.032

LLM WRITES

"Íbúum Reykjavíkur fjölgaði um 1.032 frá 1. janúar 2025 til 1. janúar 2026, úr 138.772 í 139.804."

CODE CITES

Heimild: Hagstofa Íslands, tafla MAN02005 (link to PX-Web, last-updated date)

5 The parts in depth

5.1 The catalog (built once)

`scripts/crawl_catalog.py` walks every folder of the PX-Web API (1 request per second, about an hour) and stores each table's metadata in `data/catalog.jsonl`: 2,102 entries with Icelandic and English titles, folder path, variables, every value code and its text, and the last-updated date. Numbers are not stored; they are fetched live. About 45 table ids appear twice in the catalog for different tables (e.g. MAN10001 is both "by municipality" and "by religion"), which the evaluation has to keep in mind.

5.2 Search: finding 8 candidate tables

Three independent searches each rank all 2,102 tables, and their rankings are merged. They catch different questions: exact value words, inflected Icelandic words, and meaning.

SEARCH	HOW IT WORKS	WHAT IT CATCHES
raw CODE	TF-IDF over character 3–5-grams of titles, variables, the first 40 values and English text. Accents are folded on both sides.	Exact words and values, typos, accent-less typing
lemma MIÐEIND	The same, but over base forms from BÍN: "fæddust" → "fæða", "í Kópavogi" → "kópavogur". Everyday words are expanded with official terms (SYNONYMS: verðbólga → vísitala neysluverðs).	Inflected Icelandic, everyday vocabulary
embed CODE	bge-m3 turns the question and each table's text into 1,024 numbers; tables whose vectors point the same way as the question's rank high.	Meaning without shared words ("verðbólga" ~ "vísitala neysluverðs")

> **Concept: TF-IDF.** Each table becomes a list of weighted terms. A term counts more when it is frequent in that table but rare across all tables, so "kópavogur" matters more than "fjöldi". The vectors are scaled to the same length, which is why indexing 63 town names in a table *diluted* its topic words (see the experiment log).

> **Concept: Reciprocal rank fusion (RRF).** Instead of adding scores that aren't comparable, each search gives every table its rank, and the fused score is Σ 1 / (60 + rank). A table ranked high by any one search ends up high. A weak member drags the result down, so a new search model must be measured inside the fusion, not alone.

After the fusion come three corrections:

- **Archive weighting:** older tables (eldra efni, Sögulegar, census) are multiplied by 0.6, because current tables usually answer better.
- **Place boost** (`Places`): if the question names a municipality, tables containing it are multiplied by 1.5 in lemma and 1.2 after fusion. Official and plain names are linked through shared 4-digit codes (Kópavogsbær = Kópavogur), learned from the catalog itself.
- **Checker** (`checker.py`): within the top 20, tables that lack the place, country or year asked about move below those that have them. Places are recognised with BÍN's place categories.

Before all this, **spelling is corrected** with GreynirCorrect (skipped for English, which it would turn into Icelandic; decided by a small detector that was right on all 810 questions it was checked on). The checker only sees capitalised names, and correction capitalises "kópavogi", so the two help each other.

> **Concept: time complexity of the search.** The embedding search compares the question vector with every table: O(N·d), about 2 million multiplications, roughly a millisecond. Approximate vector indexes (FAISS, HNSW) reach O(log N), but that only pays off at millions of vectors. Here the time goes elsewhere (section 9).

5.3 From table to number

1. **Pick the table** LLM: the model sees the 8 candidates' titles and variable names and returns one index as JSON. If none fits, that part is reported as "no data found".
2. **List the options** CODE: every variable with its codes and texts; lists over 60 values are trimmed to what looks relevant plus the first and last ones.
3. **Choose values** LLM: JSON with value codes. For comparisons the prompt asks for every value needed (both years; the part and the whole).
4. **Validate** CODE: codes that don't exist in the table are dropped, so the model cannot invent values; required variables it skipped get defaults (the latest period, or the total).
5. **Fetch** CODE: POST to PX-Web, json-stat2 format, 0.4 s between calls, retries on busy servers, timeouts and network errors. The result is flattened into rows.

5.4 Calculations and two-table questions (new, not yet measured)

- **Calculations**: when a part returns two or more rows, the model names *one* operation from a fixed list (difference, % change, share, ratio, sum, average) and which rows. `calculate()` checks that the rows exist and that nothing is divided by zero, and computes. The answer prompt receives the result marked "computed in code, use as is".
- **Two tables**: `decompose()` lets the model split a question about two different things ("population and inflation") into two self-contained questions. Each runs search and numbers on its own; the answer combines them and names each table and period. One-table comparisons are not split.

Both are unit-tested with a fake model and a fake PX-Web (40 tests pass), and 12 new test questions with known answers were added (7 calculation, 5 two-table). They have not yet been run with a real model.

5.5 The answer and the citation

The model writes one answer from the fetched rows only, in the language of the question, naming the period and unit. Then code adds the citation: table id, title, a link to the table on PX-Web and its last-updated date, one line per table used. Because code adds it, the source cannot be hallucinated.

6 The language models

Measured on gpu-talva (a home PC with a 4 GB GPU; most of each model ran on the CPU) with 24 questions that have known answers. "Oracle" gives the model the right table, so it measures the model alone; "full" is the whole pipeline. With 24 questions each one is ~4 points, so differences under ~10 points are noise.

MODEL	THINKING	PICK	ORACLE NUMBER / VALUES	FULL NUMBER / VALUES	SECONDS PICK / ORACLE / FULL
qwen3:4b-instruct	none	17%	62 / 54	17 / 8	4 / 17 / 7
gemma4:e2b	on	46%	58 / 54	42 / 21	14 / 22 / 33
gemma4:e2b	off	33%	29 / 29	17 / 8	3 / 6 / 5
ministral-3:8b	none	50%	88 / 88	62 / 46	10 / 39 / 25
gpt-oss:20b	low	42%	67 / 67	62 / 33	16 / 38 / 46
gemma4:26b	on	58%	100 / 100	83 / 54	179 / 259 / 420
gemma4:26b	**off**	46%	**100 / 96**	71 / 42	**13 / 32 / 18**

- **Choice: gemma4:26b with thinking off.** 14× faster than with thinking, and just as accurate on values and numbers. Thinking only helped table picking (fewer "no table fits" refusals). It is Apache-2.0, a mixture-of-experts model with ~4B active parameters, so it is fast on suitable hardware.
- **Size matters more than family:** the small models' main failure is wrong values, above all the year.
- **No invented numbers** anywhere: every miss with right values was a value miss.
- **Problems seen:** wrong units ("277.511 kg" of cod; the data is in tonnes), English questions answered in Icelandic, and clumsy Icelandic in the smaller models.
- **Hagur** (Hagstofa's own `hagstofa-ai:latest`, Llama 3 + LoRA, distributed through Ollama) will be compared at the event. A possible split: gemma4:26b for the JSON steps, Hagur for the wording if its Icelandic is better.
- **Miðeind** has no open chat model of this kind; its tools are used around the LLM (BÍN, GreynirCorrect, Greynir). Its Icelandic encoder IceBERT is a candidate search model after fine-tuning (section 8).

7 How we measure

QUESTION SET	SIZE	PURPOSE
hand	36	Hand-written; SYNONYMS were tuned on it, so it overstates
dev	270	Generated for 299 sampled current tables; split from test *by table*; tune on this
test	627	Frozen: run rarely to report, never change anything because of it

municipalities	24	Questions naming towns (the place boost was tuned on it)
real	11	Atli's own questions, typos kept: the fairest test we have
misspelled dev	540	Dev without accents and with typical typos
e2e	36	Questions with known answers: 24 easy, 7 calculation, 5 two-table

- **Retrieval:** is the right table 1st, in the top 3, in the top 8? "Right folder" also shows when search lands in the right subject area (generated questions list only their source table, so scores are pessimistic).
- **End to end:** the expected number is fetched live from PX-Web with the known value codes, and the answer must contain it (Icelandic or English number formatting, 0.5% tolerance). For calculations it is computed with the pipeline's own code; for two-table questions every number must appear.
- **Model comparison** (`compare_models.py`): pick, oracle and full per model, with seconds per question; every answer is saved so the Icelandic can be read.

> **Concept: why dev and test are split by table.** If questions about the same table were on both sides, a fix made while looking at a dev question could also "teach" the test question about that table, and the test score would overstate. With ~270 questions, ~3 points is noise; don't pick a winner on 1–2.

8 Experiment log

WHEN	IDEA	RESULT	DECISION
22–23 Oct	Skeleton; BÍN lemmas; archive down-weighting	First pipeline and lemma search	Kept
24 Oct	Value-label boost (boost tables whose values contain question words)	No gain in any form; 173 tables contain "kópavogur"	Rejected
24 Oct	Held-out sets (897 generated questions, dev/test by table)	A fair way to decide	Kept
24 Oct	Three-way fusion raw + lemma + embed	Dev top 1/3/8: 42/66/82 (best single: 36/54/72)	Kept
24 Oct	Municipality names, round 1: index official + plain names and all towns	No gain: town names diluted topic words (TF-IDF length normalisation)	Rejected
24 Oct	Place boost (round 2): a separate signal instead of text	Municipality set 8/21/58 → 54/71/88; dev unchanged	Kept
24 Oct	Spelling correction + accent folding	No accents 33/58/73 → 40/63/77; typos 41/64/82 → 44/66/82; English protected	Kept
24–25 Oct	LLM table picking with qwen3:4b; forced top-3 choice; retriever agreement	28% vs search's 42%; forced 41%; all three agree → #1 right 87%	Not with a 4B model
25 Oct	Value checker (places, countries, years)	Dev 42/66/82 → 51/71/87; test 44/66/83 → 50/70/85	Kept (default)
27 Oct	Merge checker + place boost + spelling	Dev 51/71/86; municipality 38/58/79 → 54/88/96; misspelled 41/65/81 → 47/69/85; real 36/36/55 → 45/45/64	Kept
27–29 Oct	Model comparison (section 6)	gemma4:26b, thinking off: oracle 100%	Chosen

29 Oct	Miðeind's IceBERT as search model, untrained	Embed alone 7/13/20 (bge-m3 30/50/65); inside fusion it hurts	Only after fine-tuning
29 Oct	Calculations and two-table questions	Built and unit-tested	To measure

9 Hardware and speed

Each answer needs up to five LLM calls (split, pick, values, operation, wording). An LLM call has two phases: **reading the prompt** (prefill; our value prompt is 2,000–4,000 tokens) and **writing** (decode; answers are short). On gpu-talva the model reads at ~17 tokens/s and writes at ~6.8 tokens/s, because most of it runs on the CPU: reading one long prompt takes minutes.

PART OF AN ANSWER	TIME
Search (the O(N·d) part)	~1 ms
Embedding the question, spelling correction, PX-Web query	~0.5 s each (less when cached)
LLM calls	**seconds to minutes each: ~95% of the total**

> **Concept: Amdahl's law.** Speeding up a part only helps in proportion to its share of the total. Making the search 1,000× faster saves a millisecond; making the LLM calls faster or fewer saves almost everything. Measure first, then optimise the biggest part.

DGX Spark vs LUMI. The Spark is one machine with an NVIDIA GPU and 128 GB of shared memory: Ollama and Hagur run directly, prompts are read on the GPU at thousands of tokens/s, and gemma4:26b should answer in seconds (an estimate, to be measured). LUMI is a supercomputer with AMD GPUs reached through an SSH tunnel and a job queue: excellent for heavy work (generating training questions, fine-tuning, re-embedding), but the tunnel adds risk to a live demo. Plan: the chatbot on a Spark if we get one (only 6 exist; Atli is asking), heavy work on LUMI. Our code switches with one variable.

Further speed-ups, by gain: time every step (also needed for the demo); skip the LLM table pick when all three searches agree; skip the split for clearly single-topic questions; run two-table parts in parallel; stream the answer.

10 Why Track 2

Track 2's focus, "data selection and backend logic", is exactly what we built: finding the right table among all of them, choosing and validating values, live PX-Web queries, and calculations in code. Their Track 2 starter handles three tables with keywords and no LLM; ours handles the whole catalog with measured accuracy. Track 1's prepared data covers only migration and the CPI, so it can't answer most of the test questions. From Track 3 we take one element: fine-tuning the search model on LUMI.

Search-model fine-tuning (prepared, Track 3-style): an LLM writes 5 questions per table for the ~950 current tables that are in no evaluation set, then bge-m3 and Miðeind's IceBERT are trained contrastively (question and its table pulled together, other tables in the batch pushed apart) and measured on dev. The language model itself is not fine-tuned: numbers must come live, and given the right table it is already right every time.

11 Where we stand and what is missing

JUDGING CRITERION	STATUS	GAP
Functionality	Strong and measured (search 86% top 8; values 100% given the table)	Calculations and two-table questions unmeasured; English answers
Presentation	Plain Gradio chat	FastAPI server; values used, units and response time shown
Usability	Accepts typos and missing accents	Stated assumptions for unclear questions; example questions
Clarity	Source added by code	Units from the data, so the model can't write "kg" for tonnes
Deployment	Pinned requirements, setup script, 40 tests	Running on the Spark or through the LUMI tunnel, untested
User value	—	A clear story: who is this for?

Smaller items: forced table choice (no "none fits" from the model), Icelandic number formatting in code, a Greynir safeguard on answers, villages that aren't municipalities (Stokkseyri, Kópasker), a time resolver for "í fyrra", and an adapter so the organizers' `gagnvist evaluate` scores our pipeline for the write-up. The newest code is on the branch, not yet on `main`.

12 Plan, split and timeline

	ATLI: LANGUAGE MODEL AND ICELANDIC	SOUKAÏNA: INFRASTRUCTURE AND ENGINE
Owns	Prompts, model comparison and choice, Icelandic quality, units and stated assumptions, time expressions	Model serving (Spark/LUMI), FastAPI server and deployment, retrieval and evaluation mechanics, security, English and other languages
Metric	Right values and number given the right table	Right table in the top 8, response time, uptime

WHEN	WHAT
Before Friday	SSH key in MyCSC (LUMI), accept the GitHub invitation, Python 3.11+, Git, 6 GB free disk; check e-mail from CSC; ask for a DGX Spark.
Fri 30 Oct 15:00	Kickoff and data release. Automatic check from 15:30: read Slack and the starter kit, run our search on their 52 questions against embeddings alone.
Sat 31 Oct	Morning: serve gemma4:26b on the Spark or LUMI, compare with Hagur, end to end by noon. Afternoon: FastAPI, units, values shown, assumptions; fine-tuning on LUMI if time allows.
Sun 1 Nov	Final numbers with `gagnvist evaluate`, write-up, 5–7 slides, rehearse. Presentations 20:00.

13 Lessons learned

- **A metric is code too.** gpt-oss looked terrible (25%) until the raw answers showed a narrow no-break space in "394 324" that the scorer didn't read. Read raw answers before believing a surprising number.
- **More text can make search worse.** Indexing all town names diluted topic words; a separate signal (the place boost) worked.

- **One variable at a time.** Comparing thinking models with a non-thinking one mixed two effects (a confounding factor); measuring thinking on and off separated them.
- **Find where errors happen.** The oracle/full split showed values are solved and table picking is the weak step, so effort goes there.
- **Measure before optimising.** The search is milliseconds; the LLM calls are the time.
- **Code for what must be exact.** Arithmetic, codes and sources in code; judgement and language in the model.

14 Glossary

TERM	MEANING
PX-Web, json-stat2	Hagstofa's database API and the JSON format it returns tables in
Embedding (vector)	A list of numbers that represents meaning; similar texts get similar vectors
TF-IDF	Weighting of terms: frequent in this document, rare in others
Lemma (BÍN)	A word's base form; BÍN is Árnastofnun's inflection database, used through Miðeind's tools
RRF	Reciprocal rank fusion: merging rankings by Σ 1/(60 + rank)
Top 1 / 3 / 8	The right table is first / in the first 3 / in the first 8
Oracle vs full	The model given the right table vs the whole pipeline including search
Thinking (reasoning)	Hidden reasoning a model writes before answering; slower, sometimes better
MoE	Mixture of experts: a large model of which only a part is used per word, so it is fast
LoRA	A small trained add-on that adjusts a large model cheaply (how Hagur was made)
Prefill / decode	The two phases of an LLM call: reading the prompt, writing the answer
Contrastive fine-tuning	Training a search model by pulling a question and its table together and pushing others apart
Dev / test split	Tune on one set, confirm once on another the changes were never tuned on

15 Where things are in the code

About 940 lines of core code; the rest is measurement, scripts and tests.

FILE	LINES	WHAT IT DOES
`app.py`	37	Chat UI (Gradio); becomes the FastAPI server
`hagstofa/llm.py`	49	Connection to any OpenAI-compatible LLM server
`hagstofa/px.py`	72	PX-Web: metadata, queries, pacing, retries
`hagstofa/pipeline.py`	257	Split, pick, values, validation, calculation, answer
`hagstofa/retrieve.py`	364	Search: raw, lemma, embed, fusion, Places, spelling
`hagstofa/checker.py`	83	Moves down tables lacking the asked place or year
`hagstofa/icelandic.py`	76	Miðeind tools: GreynirCorrect, Greynir
`eval/run_eval.py`	175	Retrieval and end-to-end evaluation
`eval/compare_models.py`	182	Model comparison: pick, oracle, full

`scripts/finetune_*.py`	242	Search-model fine-tuning for LUMI
`lab.py`	205	The lab UI for exploring each step
`tests/`	~390	40 tests: how each part is meant to behave

A Appendix: the full diagram

Also delivered as its own large-format PDF (system-diagram.pdf) for reading at full size.

